Qué hace el script

Mi planteamiento del script es el siguiente:

1. Antes de iniciar a trabajar con los datos, crear las propiedades de vies_ruben_estado... y el property group en Companies.
2. **Buscar en la API de HubSpot** y devolver sólo las empresas que tengan un **nif** y que no se les haya asignado ningún estado o sea ERROR_TEMPORAL.
3. **Validar el NIF**, antes de enviarlo a Vies verificar que el formato es correcto y cumple los requisitos del país, longitud y carácteres.
4. Llamar a la **API de Vies** para contrastar el nif y el código del país con su información sobre la empresa.
5. **Clasificar** un **estado,fecha y motivo** según la respuesta de Vies.
6. **Escribir** en **Hubspot** en las propiedades estado,fecha,motivo en las empresas.
7. Mostrar el resumen por consola y guardarlo también en un archivo .txt

Al principio comencé haciendo peticiones con **limit** para aprender a usar la API de HubSpot y de Vies, también para poder ir probando lógica sin hacer grandes requests.

Decisiones Técnicas

Antes de Programar

Para la **Service Key** he elegido los permisos que considero estrictamente necesario que son de leer y escribir empresas y escribir las propiedades. **Read** era necesario para acceder a las empresas y **write** para poder crear las properties y asignarles un valor.

crm.objects.companies.read **crm.schemas.companies.read**
crm.objects.companies.write **crm.schemas.companies.write**

Para la **API** de **Hubspot** he elegido la 2026-09 ya que en su documentación aparece como Latest y la v3 es Legacy, he preferido usar una versión que no estuviese en Beta o deprecada.

Código

He usado **Guzzle** para gestionar las peticiones API, me parece más legible y fácil de utilizar que cURL, también me ha servido para poder gestionar yo los códigos HTTP desactivando http_errors.

Empresas de Hubspot: Para hacer fetch de las empresas correctas y no repetidas he filtrado obligatoriamente que tuviesen un nif y que no tuviesen un Estado(Valido,Invalido etc) o que el estado fuese ERROR_TEMPORAL.

Para la **paginación** entrego **after** en el body de la petición que me lo saco a partir de la primera respuesta de la API, así se ejecuta hasta que recorre todas las páginas.

Los **Rate Limit** los gestiono configurando Retry-After por si se queda colgado y si por esta razón u otro motivo se queda colgada la petición el estado se guarda como ERROR_TEMPORAL para volver a ser clasificado.

API Vies

- Para hacer peticiones tuve que usar **POST** a "check-vat-number", porque con GET devuelve 405, pasándole 2 parámetros el código del país y el nif.
- Para la fecha de validación **sólo** uso la fecha de las **peticiones válidas** que devuelve VIES, no uso una génerica para todas las peticiones, ya la devuelve en formato ISO8601.
- Los errores de Vies son **difíciles** de **gestionar**. Dependiendo del error puede devolver un código **HTTP 200**, pero en el body un error y succeed = false, por lo que hay que gestionar los códigos **Y** el body para gestionar los errores.

Gestión de datos

Para los **caracteres,** uso un regex que solo permite carácteres de la A a la Z y números de 0 a 9, para ahorrar posibles formatos incorrectos en una petición a Vies prefiero gestionarlo desde el principio.

El prefijo del país lo contrasto con un array con los códigos de los países miembros de la **Unión Europea**, si no es ninguno de estos no se acepta. Para **Grecia**, si hay un código que contenga GR lo transformo a **"EL"** que es el código aceptado.

La **longitud** del código sin prefijo la dejo entre **2 y 12** ya que cada país tiene una longitud distinta.

Arquitectura

He mantenido **una responsabilidad por clase** y asignando las dependencias por constructor, para que sea más mantenible y escalable. También de esta forma es mucho más factible ejecutar Tests Unitarios para cada clase.

Para devolver datos he hecho múltiples **DTOs** (ApiResponse,NormalizedVat) para devolver valores de forma regular y es más fácil y ordenado el acceso a los datos.

Modo de prueba con --dry-run, si se ejecuta el script en modo prueba se lleva a cabo toda la funcionalidad excepto escribir en Hubspot.

Por último saco el resultado tanto por consola como por archivo .txt, para poder verlo en vivo y también que haya permanencia del informe.

Notas Personales

- Encontrar la documentación de la API de Vies ha sido difícil y no aparecían los códigos de error.
- Vies responde con códigos 200 aunque devuelva un error, esto me hizo más complejo la lectura de errores.
- Para un país sin código lo descarto, y para Grecia convierto el código, no sé si esto es lo esperado, es la decisión que tomé.

Con más tiempo

- Haría **tests** con **Pest.**
- Añadir más **comentarios** al código
- Hacerlo más **modular**.
- Exportar **JSON** para poder exportar a otras aplicaciones.

Rubén Camús - Escrito a 🖐